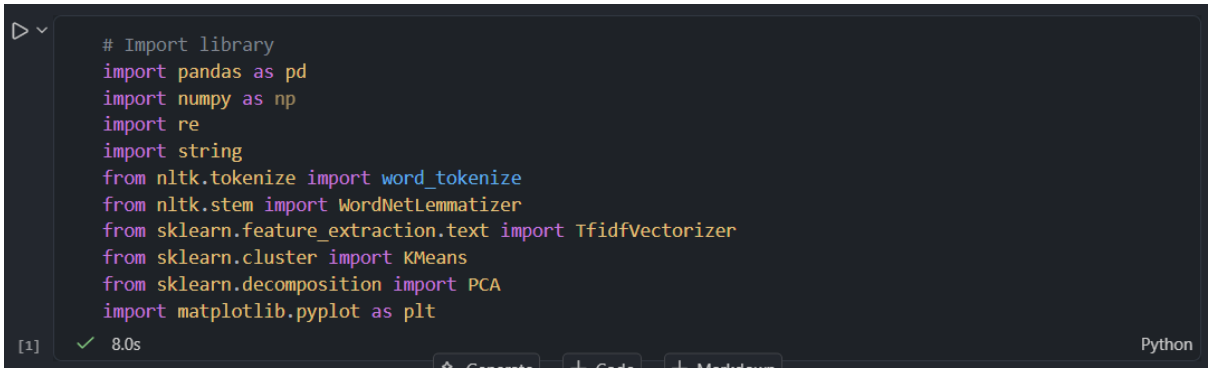
LAPORAN PRAKTIKUM
KECERDASAN BUATAN



RUBEN SIBARANI
NIM: 42324037

INSTITUT TEKNOLOGI DEL
D3 TEKNOLOGI INFORMASI



import re
Modul *Regular Expression* bawaan Python yang digunakan untuk pencarian, pencocokan, dan pembersihan pola string (misalnya untuk menghapus angka atau simbol khusus dari teks).

import string
Menyediakan sekumpulan konstanta string bawaan, seperti daftar tanda baca (*punctuation*), yang sering digunakan untuk membersihkan teks dari simbol-simbol yang tidak diperlukan.

from nltk.tokenize import word_tokenize
Berfungsi untuk melakukan tokenisasi, yaitu memecah kalimat atau teks panjang menjadi potongan-potongan kata individual (token).

from nltk.stem import WordNetLemmatizer
Digunakan untuk lemmatisasi, yaitu proses mengubah suatu kata kembali ke bentuk dasar atau kamusnya (*lemma*) berdasarkan kamus linguistik WordNet.

from sklearn.feature_extraction.text import TfidfVectorizer
Berfungsi untuk mengubah teks mentah menjadi matriks fitur numerik menggunakan metode TF-IDF, sehingga komputer dapat memahami bobot pentingnya suatu kata dalam dokumen.

from sklearn.cluster import Kmeans
Algoritma *Machine Learning* berbasis *unsupervised learning* yang digunakan untuk melakukan klasterisasi (mengelompokkan data teks ke dalam beberapa klaster berdasarkan kemiripannya).

from sklearn.decomposition import PCA
Digunakan untuk Reduksi Dimensi. Karena hasil vektor TF-IDF memiliki dimensi yang sangat banyak, PCA mereduksinya menjadi lebih sedikit dimensi agar data dapat divisualisasikan dalam bentuk grafik 2D.

import matplotlib.pyplot as plt
Digunakan untuk membuat berbagai macam grafik dan visualisasi data, yang pada kode selanjutnya dipakai untuk menggambar sebaran titik klaster hasil *K-Means* dan *PCA*.

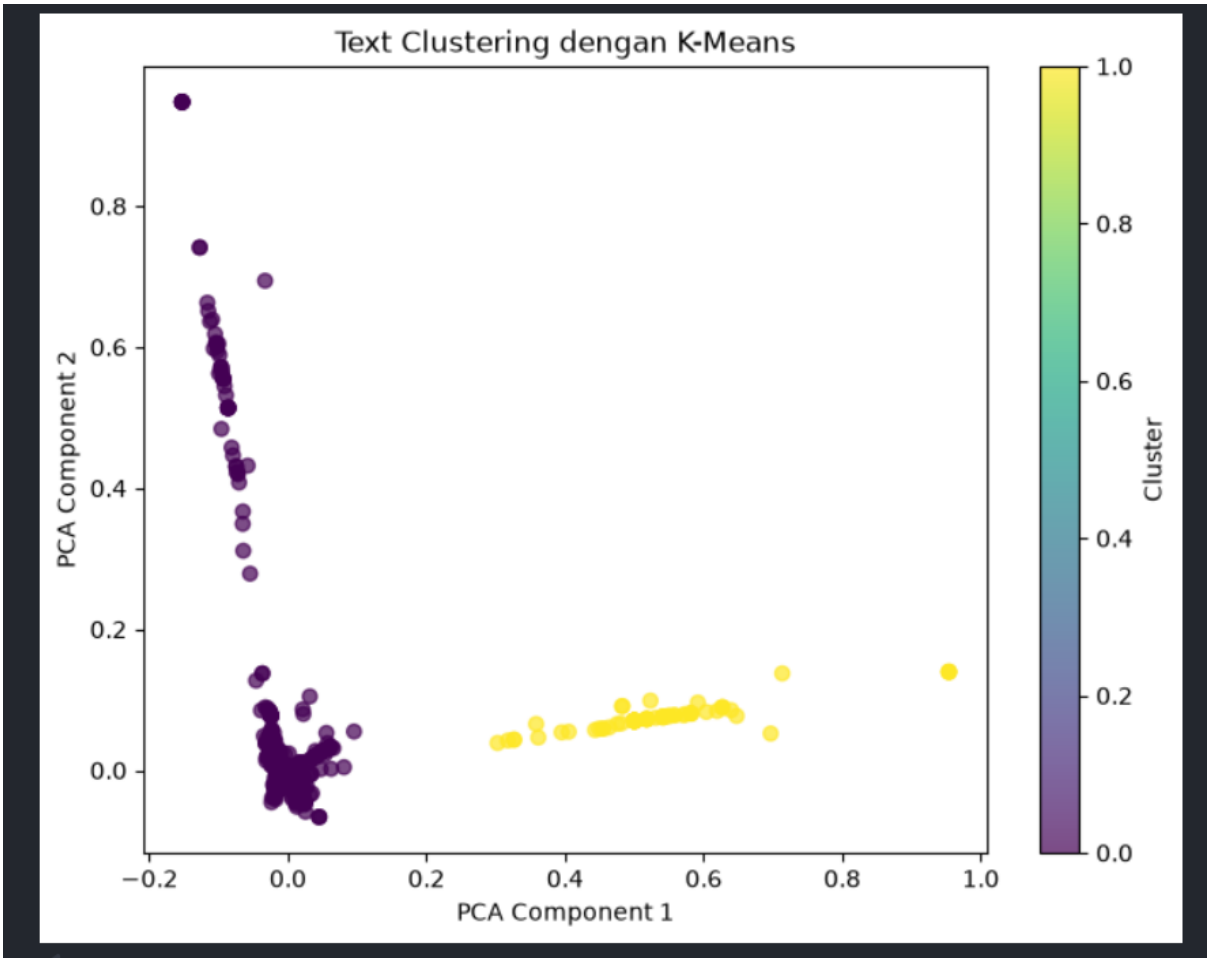
```
# Load dataset
file_path = "tmdb_5000_movies.csv"
df = pd.read_csv(file_path)
# Hanya mengambil kolom message untuk klasterisasi
df = df.rename(columns={'title': 'goar'})
```

[4] ✓ 0.2s Python

kode tersebut berfungsi untuk memuat dataset ke dalam DataFrame Pandas dan mengubah nama kolom 'title' menjadi 'goar'

```
# Inisialisasi lemmatizer
lemmatizer = WordNetLemmatizer()
# Fungsi preprocessing teks
def clean_text(text):
    text = text.lower() # Lowercasing
    text = re.sub(r'\d+', '', text) # Hapus angka
    text = text.translate(str.maketrans('', '', string.punctuation)) # Hapus tanda baca
    words = word_tokenize(text) # Tokenisasi
    words = [lemmatizer.lemmatize(word) for word in words] # Lemmatization
    return ' '.join(words)
# Terapkan preprocessing ke teks
df['clean_text'] = df['goar'].apply(clean_text)
# Konversi teks ke vektor TF-IDF
vectorizer = TfidfVectorizer(stop_words='english') # Menggunakan stopwords dari sklearn
X = vectorizer.fit_transform(df['clean_text'])
# Tentukan jumlah klaster
num_clusters = 2
kmeans = KMeans(n_clusters=num_clusters, random_state=42, n_init=10)
df['cluster'] = kmeans.fit_predict(X)
# Visualisasi dengan PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X.toarray())
plt.figure(figsize=(8, 6))
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=df['cluster'], cmap='viridis', alpha=0.7)
plt.xlabel("PCA Component 1")
plt.ylabel("PCA Component 2")
plt.title("Text Clustering dengan K-Means")
plt.colorbar(label='Cluster')
plt.show()
```

[5] ✓ 1.6s Python



lemmatizer = WordNetLemmatizer(): Menyiapkan alat untuk mengembalikan kata berimbuhan atau jamak ke bentuk kata dasarnya.

text.lower(): Mengubah seluruh huruf pada teks menjadi huruf kecil agar kata yang sama tidak dibedakan oleh kapitalisasinya.

re.sub(r'\d+', '', text): Menghapus seluruh karakter angka atau digit yang terdapat di dalam teks.

text.translate(...): Menghapus semua tanda baca (*punctuation*) seperti titik, koma, tanda seru, dan simbol lainnya.

word_tokenize(text): Memecah kalimat atau judul menjadi potongan-potongan kata terpisah (*token*).

[lemmatizer.lemmatize(word) for word in words]: Mengubah setiap kata ke bentuk dasarnya berdasarkan kamus linguistik.

return ' '.join(words): Menggabungkan kembali daftar kata yang sudah bersih menjadi satu kalimat utuh.

Penerapan Preprocessing ke Dataset
df['clean_text'] = df['goar'].apply(clean_text): Menjalankan fungsi clean_text ke setiap baris pada kolom dataset (misalnya kolom 'goar') dan menyimpannya ke kolom baru bernama 'clean_text'.

Konversi Teks ke Vektor TF-IDF
TfidfVectorizer(stop_words='english'): Menyiapkan metode pembobotan kata TF-IDF sekaligus membuang kata-kata umum dalam bahasa Inggris yang kurang memiliki makna penting (seperti *the, is, an*).

X = vectorizer.fit_transform(...): Mengubah seluruh teks bersih menjadi matriks angka (*vektor*) agar dapat dipahami dan diproses oleh komputer

Klasterisasi Menggunakan K-Means
num_clusters = 2: Menentukan jumlah kelompok atau klaster yang ingin dibentuk, yaitu sebanyak 2 klaster.
KMeans(...): Menginisialisasi algoritma klasterisasi *K-Means* dengan titik acak awal yang konsisten (random_state=42).

df['cluster'] = kmeans.fit_predict(X): Melatih model dengan data angka dan langsung memberikan label nomor klaster (misalnya 0 atau 1) untuk setiap baris data.

Reduksi Dimensi dengan PCA & Visualisasi Grafik

PCA(n_components=2): Meringkas matriks fitur teks yang memiliki dimensi sangat banyak menjadi dua komponen utama (untuk sumbu X dan Y).

X_pca = pca.fit_transform(...): Mengubah data vektor berdimensi tinggi menjadi array 2 dimensi (X_pca).

plt.scatter(...): Menggambar diagram sebaran titik (*scatter plot*) di mana posisi titik ditentukan oleh komponen PCA dan warnanya dibedakan berdasarkan hasil klaster.

plt.colorbar(...): Menampilkan batang penunjuk gradasi warna klaster di sisi kanan grafik untuk mempermudah pembacaan.

plt.show(): Menampilkan hasil akhir visualisasi grafik ke layar.